

Working with Data

Session 1 — What is this data?

Block 2 — One table, one question: SQL on DuckDB

This block in one sentence

A number is a sentence before it is a query: *a measure, over a population, after a filter.*

SQL is that sentence written precisely enough for a machine to answer it and for a colleague to check it.

NULL is waiting at every clause.

Agenda

1. Why SQL: 525,461 lines, watched
2. The map: the order a query runs
3. The number as a sentence · the aggregates · `NULL` in them
4. `GROUP BY`, the `NULL` group, the sum identity, a share of a total
5. `HAVING` against `WHERE`
6. Filters and `NULL`, mechanised · calculated columns · dates
7. Lab 2 — on the other file

Why SQL — watch

On my screen, the whole Online Retail file, **525,461 invoice lines**, in DuckDB:

```
import time
t = time.perf_counter()
lines = con.sql("SELECT COUNT(*) FROM 'data/raw/online_retail.parquet'").fetchone()[0]
by_country = con.sql("""
    SELECT Country, SUM(Quantity * Price) AS revenue
    FROM 'data/raw/online_retail.parquet'
    GROUP BY Country
    ORDER BY revenue DESC
""").df()
print(f"{lines:,} lines, {len(by_country)} countries, {time.perf_counter() - t:.3f} seconds")
```

The same file in Excel — watch

The same lines as a CSV: **46,810,498 bytes (46.8 MB)**. As Parquet: **3,218,445 bytes (3.2 MB)**. DuckDB reads either as a table.

On my machine, today, live: open the CSV in Excel; add a revenue column; a pivot of revenue by country. **The time is whatever the clock says.**

525,461 rows is under Excel's documented limit of 1,048,576 rows a sheet. The question is not *can it*. It is:

- **Can a colleague check it?** A query is text they can read and rerun. A pivot is clicks.
- **Next month's file?** One line, rerun. The clicks, again.
- **What does it do with a mistake?** DuckDB refuses to turn text it cannot read into a number. A spreadsheet guesses.

Parquet, in one slide

	CSV	Parquet
the same 525,461 rows	46.8 MB	3.2 MB
stored	row after row, as text	column by column, compressed
types	guessed every time it is read	stored with the data

`Customer ID` is still a `DOUBLE` in the Parquet: the type the CSV reader guessed was saved with it. That is all this course says about Parquet.

The lecture file: one row is one country in one year

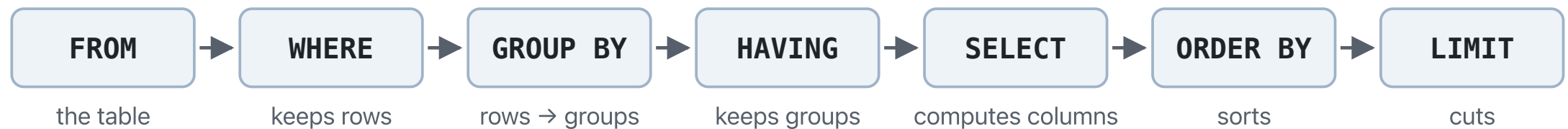
Watch (cell 1 of `lecture.ipynb`): Block 1's key test, on the lecture file.

```
con.sql("""
    SELECT COUNT(*)                AS n_rows,
           COUNT(DISTINCT (country, year)) AS n_keys,
           COUNT(DISTINCT country)      AS countries
    FROM 'data/raw/countries.csv'
""").df()
```

219 countries, 1990 to 2024. One row per country per year?

The map: the order a query runs

The order it runs



The order you write it: SELECT · FROM · WHERE · GROUP BY · HAVING · ORDER BY · LIMIT

Each clause gets the result of the one before it. Nothing can use what has not been made yet.

You write **SELECT** first. It runs **fifth**.

The sentence first, then the query

Total CO₂ emitted by countries in 2024 is the sum of **co2** (the measure) **over the country rows** (the population) **of 2024** (the filter).

Predict, then watch (cell 2):

```
con.sql("""
    SELECT SUM(co2) AS total_co2
    FROM 'data/raw/countries.csv'
    WHERE year = 2024
""").df()
```

Predict: how many rows come back? One per country, or one in all?

Seven aggregates, and what **NULL** does to them

Predict, then watch (cell 3). Predict: are **n_rows** and **n_co2** equal?

```
con.sql("""
    SELECT COUNT(*) AS n_rows, COUNT(co2) AS n_co2, COUNT(DISTINCT country) AS n_countries,
           SUM(co2) AS total_co2, AVG(co2) AS avg_co2, MIN(co2) AS min_co2, MAX(co2) AS max_co2
    FROM 'data/raw/countries.csv'
    WHERE year = 2024
""").df()
```

An average with the missing left out, and with them as zeros

Predict, then watch (cell 4):

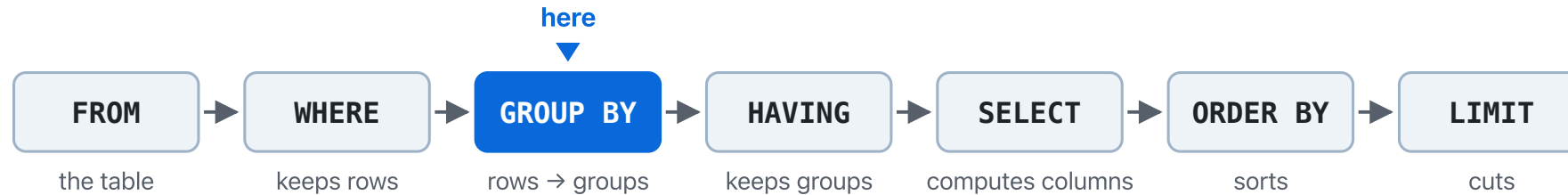
```
con.sql("""
    SELECT AVG(gdp)                AS avg_gdp,
           AVG(COALESCE(gdp, 0)) AS avg_gdp_zero
    FROM 'data/raw/countries.csv'
    WHERE year = 2022
""").df()
```

`COALESCE(gdp, 0)` returns the first of its values that is not `NULL` : `gdp` where there is one, otherwise `0`.

Predict: the same number twice, or two different ones? If different, which is larger? (The output prints `e+11` : $\times 10^{11}$.)

GROUP BY: one row per year

The order it runs



The order you write it: SELECT · FROM · WHERE · GROUP BY · HAVING · ORDER BY · LIMIT

One row per value of the grouping column. Every other column must be aggregated.

Watch (cell 5). One row per year.

```
con.sql("""
    SELECT year, SUM(co2) AS total_co2
    FROM 'data/raw/countries.csv'
    GROUP BY year
    ORDER BY year
""").df()
```

The rule **GROUP BY** enforces

Predict, then watch (cell 6; it is wrong on purpose). Predict: does DuckDB pick a country, or refuse?

```
SELECT year, country, SUM(co2) FROM 'data/raw/countries.csv' GROUP BY year
```

The **NULL** group

Predict, then watch (cell 7). Predict: how many rows — two, or three?

```
con.sql("""
    SELECT gdp > 1000000000000 AS big_economy, COUNT(*) AS countries, SUM(co2) AS co2
    FROM 'data/raw/countries.csv'
    WHERE year = 2022
    GROUP BY big_economy
    ORDER BY big_economy
""").df()
```

The sum identity: a breakdown adds up to its total

Predict, then watch (cell 8). Predict: do *big* plus *not big* add up to the total?

Emissions are **additive**: every breakdown adds up to the total, computed by other code, or one of them is wrong. The numbers: cell 8.

Compare sums with a tolerance, never `==`. Two correct sums of the same numbers can differ in the tenth decimal. Money: to the penny, `abs(a - b) < 0.005`.

Means and ratios have their own identities. An average does not add up. A ratio's top and bottom are reconciled separately, on the same rows.

A share of a total: a query inside a query

Watch (cell 9). Each country's share of the countries' 2024 total:

```
SELECT country, co2,  
       ROUND(100 * co2 / (SELECT SUM(co2) FROM 'data/raw/countries.csv' WHERE year = 2024), 2)  
       AS pct_of_total  
FROM 'data/raw/countries.csv'  
WHERE year = 2024  
ORDER BY co2 DESC  
LIMIT 5
```

The five largest: China 32.85 · United States 13.11 · India 8.54 · Russia 4.76 · Japan 2.57.

The inner query runs once and returns one number, 37,406.117. The outer query divides every row by it.

HAVING: a condition on a group

The order it runs



The order you write it: SELECT · FROM · WHERE · GROUP BY · HAVING · ORDER BY · LIMIT

A condition on a count or a sum can only be tested once the groups exist: HAVING.

Predict, then watch (cell 10). The years in which fewer countries reported CO₂ than in 2024. Predict: could the condition go in **WHERE** instead of **HAVING** ?

```
con.sql("""
    SELECT year, COUNT(co2) AS reporting
    FROM 'data/raw/countries.csv'
    GROUP BY year
    HAVING COUNT(co2) < 216
    ORDER BY year
""").df()
```

AND, OR, and the parentheses you forgot

Predict, then watch (cell 11). The sentence: *China and India, in 2024*. Predict: how many rows does each line return?

```
WHERE year = 2024 AND country = 'China' OR country = 'India'  
WHERE year = 2024 AND (country = 'China' OR country = 'India')
```

Three more tools for WHERE, one line each: country IN ('China', 'India') · year BETWEEN 2020 AND 2024 (both ends included) · country LIKE 'United%' (% is any text).

Every country but China

Predict, then watch (cell 12). 219 countries in 2024, minus China, written two ways. Predict both counts.

```
print(con.sql("""SELECT COUNT(*) FROM 'data/raw/countries.csv'
                WHERE year = 2024 AND iso_code != 'CHN'""").fetchone()[0])
print(con.sql("""SELECT COUNT(*) FROM 'data/raw/countries.csv'
                WHERE year = 2024 AND iso_code IS DISTINCT FROM 'CHN'""").fetchone()[0])
```

NULL in a WHERE, mechanised — watch

For a row whose `iso_code` is `NULL` (Kosovo, in this table):

The condition	Its value	Kept by WHERE ?
<code>iso_code = 'CHN'</code>	<code>NULL</code> (unknown)	no
<code>iso_code != 'CHN'</code>	<code>NULL</code> (unknown)	no
<code>iso_code NOT IN ('CHN', 'IND')</code>	<code>NULL</code> (unknown)	no
<code>iso_code IS NULL</code>	true	yes
<code>iso_code IS DISTINCT FROM 'CHN'</code>	true	yes
<code>COALESCE(iso_code, 'none') != 'CHN'</code>	true	yes

WHERE keeps a row only when the condition is **true**. Unknown is not true.

Calculated columns, aliases, and division

Watch (cell 13). `co2` is in million tonnes and `population` in persons, so tonnes per person is `co2 * 1000000 / population`:

country	tonnes per person, 2024
China	8.66
United States	14.20
India	2.20

`7 / 2` is 3.5 in DuckDB. `7 // 2` is 3. `ROUND(2 / 3, 2)` is 0.67.

`AS` names a column. The name is born in `SELECT`, after `WHERE` on the map, so standard SQL does not let `WHERE` use it. DuckDB does, as a shortcut. This course writes the expression again: the query then means the same in any database.

Dates, the minimum

Watch (cells 14 and 15). One timestamp, three ways:

<code>date_trunc('month', ts)</code>	<code>year(ts)</code>	<code>strftime(ts, '%Y-%m')</code>
2010-03-01 00:00:00	2010	'2010-03'

for `ts = TIMESTAMP '2010-03-17 14:05:00'`. `date_trunc` turns a moment into **the start of its month** (or `'week'`, `'year'`): a value you can `GROUP BY`.

And a period made from a number: `year // 10 * 10 AS decade` — 4 rows, the 2020s with only 5 years in them.

Build one clause at a time — watch

1. **The sentence:** the measure, the population, the filter.
2. `FROM` + `LIMIT 10` : look at ten rows. Are they the thing?
3. `WHERE` : **count the rows** it keeps. The number you expected?
4. `GROUP BY` : **say how many groups** before you run it.
5. The aggregates. Then `HAVING` , `ORDER BY` . Remove the `LIMIT` .
6. **The check:** the identity this number permits.

What you just saw, as one idea

A filter that sounds right can throw away rows the sentence keeps.

`gdp <= 1e12` and `gdp > 1e12` lost 55 countries. `!= 'CHN'` lost Kosovo.

A ratio can be built from two different populations.

`SUM(gdp) / COUNT(*)` is a top from 164 countries over a bottom of 219: 595 billion, the same wrong number as counting the missing as zero.

No error in either. The sentence says what the number counts; the query must do all of it, and only it.

Git thread — the message names the cause

```
git commit -m "Keep countries with no GDP in the breakdown: gdp <= 1e12 dropped 55 NULL rows"
```

not `git commit -m "fix"`. Commit after each fix. Six months from now, `git log --oneline` is the only diagnosis note anyone will read.

AI thread — the assistant does not know your data

Ask an assistant: *"Average GDP per country in 2022, from this table"*, and paste `DESCRIBE`. It will write `AVG(gdp)`. Then ask it: *"over how many countries?"*

It cannot know. `COUNT(gdp)` against `COUNT(*)`: 164 against 219.

The assistant does not know your data's conventions unless you paste them. The census is how you learn them yourself.

5-minute stand-and-stretch — start the Lab 2 clone first

In your terminal (Git Bash on Windows, Terminal on macOS), in the folder where you keep course work:

```
git clone https://github.com/earino/ecbs5294-lab02-one-table.git  
cd ecbs5294-lab02-one-table  
uv sync
```

Start it, then stand up: it downloads while you stretch.

Lab 2 — The revenue the brief did not ask for

The file: `data/raw/online_retail.parquet` — the shop's invoice lines. **The brief:** finance's definitions of every number, in the README and the notebook.

The symptom: the report runs without an error and says 2010 revenue was **£8,739,637.52** and the average identified customer brought in **£2,052.52**. Its own per-customer table does not average to £2,052.52. And every number has a sentence in the brief.

First minute: the clone finished? Open the folder in VS Code, open `notebooks/report.ipynb`, pick the `.venv` kernel, run the first cell: it prints the Lab 2 folder.

Your work, in `report.ipynb`: inspect (A) → a view `sales_2010` (B) → revenue by month, from an empty cell (C) → countries with more than 100 invoices, from an empty cell (D) → **the check** (E) → **DIAGNOSIS.md**. After the note, if there is time: the per-customer figure (F, also HW1 question 7). Neighbour at minute 33. Checkpoint on Moodle.

The last ten minutes — explain it to a neighbour

One of you explains, about a minute, pointing at the screen: **what was wrong · why · the query that proved it · what you changed · how you know it is right.** The other asks:

1. Show me the query that proves it.
2. Why was it wrong, not just where?
3. A colleague says your view's filter is the long way round: ***WHERE Quantity > 0** does the same job.* Same lines? Same revenue?

Then swap. Unsure, or you disagree? **Hands up.** Then the checkpoint on Moodle.

Homework 1 assigned — One table, the weekly numbers

The head of sales wants **seven numbers** from the same file, each **a sentence, a query, a number, and a check**; two more are **stretch**, not graded. The brief carries Lab 2's definitions and a few more. Expected time: **3–4 hours**. Due **Friday 9 October, 23:59**, on Moodle.

The starter is `earino/ecbs5294-hw01-one-table` on GitHub; its README has the clone line, for home.

1. The notebook: every question in its four parts, and the **reconciliation** — by month = by country = the total, to the penny
2. **The trap log** in `DIAGNOSIS.md` : one five-part note per data issue that changed a number, even if your query was never wrong
3. Commits that name the cause · `GIT_LOG.txt` · the zip by `git archive` , as `SUBMITTING.md` says
4. A **60–90 second video**: one trap — symptom, cause, change, verification
5. `AI_USE.md`